Decentralised Lending Service - Project Report

1 Architectural Choices

The Decentralised Lending Service has been implemented using a modular and upgradeable architecture on Ethereum.

Upgradeable Pattern:
The main `FundingPool.sol` contract uses the **UUPS (Universal Upgradeable Proxy Standard)** pattern provided by OpenZeppelin. This allows the core logic to be upgraded in the future without losing the state (such as contributor balances or active proposals), while being more gas-efficient than the Transparent Proxy pattern.

Loan Management:
Instead of managing all active loans within a single massive contract, the system adopts a Factory-like pattern. Upon the approval of a loan proposal, the `FundingPool` deploys a new instance of a dedicated `LoanContract.sol`. This cleanly separates the state and logic of individual loans, reducing the complexity of the main pool and minimizing the risk of a single bug affecting all active loans.

Bitcoin Liquidity Oracle:
The cross-chain interaction is simulated via an Oracle design pattern. We implemented an on-chain `BitcoinOracle.sol` that acts as a registry. The off-chain component is a Python service (`oracle_service.py`) that syncs the Bitcoin blockchain (parsing UTXOs up to block 131,000 to maintain quick sync times) and then actively listens to `UpdateRequested` events emitted by the on-chain Oracle. When a borrower requests a balance verification, the Python script computes the real-world Bitcoin balance and pushes it on-chain, enabling the `FundingPool` to verify collateral requirements dynamically.

2 Gas Evaluation Results

During the testing phase, we profiled the main operations of the protocol to understand the gas footprint. The tests were run on the local Hardhat network.

Operation	Gas Used	Notes
`contribute()`	136,061	Initializing contributor tracking and transferring ETH.
`withdraw()`	44,929	Updating balances and processing the ETH refund.
`createLoanProposal()`	184,745	Storing a new proposal struct (heavy storage writes).
`voteOnProposal()`	79,594	Updating voting mappings and weights.
`executeApprovedLoan()`	849,759	High cost due to deploying a new `LoanContract`.
`repay()`	157,237	Transferring funds back and triggering interest distribution.

Note: The `executeApprovedLoan` is intentionally the most expensive operation because it requires deploying a new smart contract dynamically and iterating over the contributor arrays.

3 Reentrancy Vulnerability & Attack

To demonstrate security risks, we included a modified contract `ReentrancyVulnerableLoan.sol` and an attacker contract `ReentrancyAttacker.sol`.

The Vulnerability:
In the vulnerable version, the `withdraw()` function sends ETH to the user *before* updating their internal balance (`balances[msg.sender] = 0;`). This violates the Checks-Effects-Interactions pattern.

The Attack:
The `ReentrancyAttacker` contract deposits a small amount of ETH and immediately calls `withdraw()`. When the vulnerable contract sends the ETH, it triggers the attacker's `receive()` fallback function. Because the attacker's balance hasn't been set to 0 yet in the vulnerable contract's state, the fallback function can recursively call `withdraw()` again. This loop drains the contract's entire ETH balance before the state is ever updated.

Our production contracts (`FundingPool` and `LoanContract`) protect against this by strictly adhering to the **Checks-Effects-Interactions** pattern, updating state variables *before* any external ETH transfers.

4 Malicious Strategies Discussion

Exploiting the Repayment Sorting Rule
The system's requirements dictate that during a partial repayment, the base amount is split among the participants by refunding the contributors in order of their locked value, **from highest to lowest**.

This introduces a significant economic vulnerability that a malicious actor can exploit without relying on smart contract bugs (like reentrancy).

The Strategy:

1. A malicious actor deposits a massive amount of ETH into the funding pool, ensuring they are the contributor with the absolute highest disposable value.
2. The same actor applies for a very large loan. Because they have a huge disposable value, a massive chunk of their own loan will be funded by their own locked contribution, but a portion will still be funded by other honest contributors.
3. Once the loan is granted, the malicious borrower performs a **partial repayment**. Because refunds are processed from the highest locked value first, the malicious actor's partial repayment goes entirely to refunding their *own* locked contribution in the pool.
4. The malicious actor then intentionally defaults on the remainder of the loan.

Result:
The malicious actor gets all of their original contribution back (via the partial repayment), while simultaneously walking away with the remaining borrowed funds that were sourced from the honest contributors. The honest contributors take the entirety of the loss, as their locked funds were at the bottom of the refund queue.

5 User Manual

Prerequisites

- **Node.js** & **npm**
- **Python 3.10+** (with `web3` and `python-dotenv`)

- A local **Geth** node running the provided genesis block (Chain ID 202526).

1. Start the Blockchain

Navigate to the `private-chain` directory and start your Geth node:

```
./geth --datadir=./data --http --http.addr=127.0.0.1 --http.port=8545
--networkid=202526 --authrpc.port=8551 --http.api=eth,net,web3
```

2. Run Setup

Execute the Python setup script to generate accounts, deploy the contracts, and create `.env` files:

```
python python/setup.py
```

This will deploy the FundingPool and BitcoinOracle, and generate new keys for the Deployer, Contributor, and Borrower.

3. Start Background Services

Open two new terminals and start the off-chain components. Keep them running:

```
# Terminal 1: Bitcoin Oracle
python python/oracle/oracle_service.py

# Terminal 2: Auto Voter
python python/voter/auto_voter.py
```

4. Run the Simulation

In a separate terminal, run the full lifecycle simulation script:

```
python python/simulate.py
```

This script will automatically execute a deposit, propose a loan, wait for oracle and voter background services, close the voting period, execute the loan and repay it, printing all updated balances at each step.